TaskSimulator

Generated by Doxygen 1.8.6

Tue Jun 17 2014 18:28:21

Contents

1 Batch processing and Time Share simulator 1

2 Data Structure Index 3

2.1 Data Structures . . . 3

3 File Index 5

3.1 File List . . . 5

4 Data Structure Documentation 7

4.1 run_s Struct Reference . . . 7

4.1.1 Detailed Description . . . 7

4.1.2 Field Documentation . . . 7

4.1.2.1 io . . . 7

4.1.2.2 run . . . 7

4.2 task_s Struct Reference . . . 8

4.2.1 Detailed Description . . . 8

4.2.2 Member Enumeration Documentation . . . 9

4.2.2.1 anonymous enum . . . 9

4.2.3 Field Documentation . . . 9

4.2.3.1 duration . . . 9

4.2.3.2 index . . . 9

4.2.3.3 io_time . . . 9

4.2.3.4 nruns . . . 9

4.2.3.5 rtime . . . 9

4.2.3.6 run . . . 10

4.2.3.7 start . . . 10

4.2.3.8 state . . . 10

4.2.3.9 utime . . . 10

5 File Documentation 11

5.1 main.c File Reference . . . 11

5.1.1 Macro Definition Documentation . . . 12

5.1.1.1 VERSION . . . 12

5.1.2 Function Documentation . . . 12
5.1.2.1 main . . . 12
5.1.3 Variable Documentation . . . 13
5.1.3.1 quantum . . . 13
5.1.3.2 tasks . . . 13
5.2 sched.c File Reference . . . 13
5.2.1 Function Documentation . . . 14
5.2.1.1 BatchResched . . . 14
5.2.1.2 RunBatch . . . 15
5.2.1.3 RunTimeShare . . . 17
5.2.1.4 TimeShareResched . . . 19
5.3 sched.h File Reference . . . 20
5.3.1 Function Documentation . . . 20
5.3.1.1 BatchResched . . . 20
5.3.1.2 RunBatch . . . 21
5.3.1.3 RunTimeShare . . . 23
5.3.1.4 TimeShareResched . . . 25
5.4 tasks.c File Reference . . . 26
5.4.1 Macro Definition Documentation . . . 27
5.4.1.1 TASKS_C . . . 27
5.4.2 Function Documentation . . . 27
5.4.2.1 CheckTaskReady . . . 27
5.4.2.2 DisplayTasks . . . 28
5.4.2.3 ReadTasks . . . 29
5.4.2.4 StillHaveTasks . . . 31
5.4.2.5 UpdateTasks . . . 31
5.4.3 Variable Documentation . . . 32
5.4.3.1 ntasks . . . 32
5.4.3.2 tasks . . . 32
5.5 tasks.h File Reference . . . 32
5.5.1 Typedef Documentation . . . 33
5.5.1.1 run_t . . . 33
5.5.1.2 task_t . . . 33
5.5.2 Enumeration Type Documentation . . . 33
5.5.2.1 sched_t . . . 33
5.5.3 Function Documentation . . . 33
5.5.3.1 CheckTaskReady . . . 33
5.5.3.2 DisplayTasks . . . 34
5.5.3.3 ReadTasks . . . 35
5.5.3.4 StillHaveTasks . . . 36

5.5.3.5 UpdateTasks . . . 36
5.6 time.c File Reference . . . 37
5.6.1 Macro Definition Documentation . . . 38
5.6.1.1 TIME_C . . . 38
5.6.2 Enumeration Type Documentation . . . 38
5.6.2.1 anonymous enum . . . 38
5.6.3 Function Documentation . . . 39
5.6.3.1 DisplayStats . . . 39
5.6.4 Variable Documentation . . . 39
5.6.4.1 cpu_end . . . 39
5.6.4.2 cpu_time . . . 40
5.6.4.3 quantum . . . 40
5.7 time.h File Reference . . . 40
5.7.1 Function Documentation . . . 40
5.7.1.1 DisplayStats . . . 40

Index **42**

Chapter 1

Batch processing and Time Share simulator

Author

Xavier Bonnaire

Copyright

Universidad Técnica Federico Santa María - 2014

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

run_s 7
task_s 8

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

main.c 11
sched.c 13
sched.h 20
tasks.c 26
tasks.h 32
time.c 37
time.h 40

Chapter 4

Data Structure Documentation

4.1 run_s Struct Reference

`#include <tasks.h>`

Data Fields

- unsigned int run

  The time before doing an IO.

- unsigned int io

  The IO duration. If 0 then end of task.

4.1.1 Detailed Description

Definition at line 4 of file tasks.h.

4.1.2 Field Documentation

4.1.2.1 unsigned int run_s::io

The IO duration. If 0 then end of task.

Definition at line 6 of file tasks.h.

Referenced by ReadTasks(), RunBatch(), and UpdateTasks().

4.1.2.2 unsigned int run_s::run

The time before doing an IO.

Definition at line 5 of file tasks.h.

Referenced by ReadTasks(), RunBatch(), and RunTimeShare().

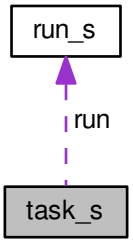
The documentation for this struct was generated from the following file:

- tasks.h

4.2 task_s Struct Reference

`#include <tasks.h>`

Collaboration diagram for task_s:



Public Types

- enum {
  ready, blocked, terminated, notready,
  running }

Data Fields

- unsigned start

  The task arrival time.
- unsigned duration

  The task duration (total)
- int utime

  The task user time.
- int rtime

  The task elapsed time (respond time)
- run_t run

  Array of run_s.
- unsigned nruns

  Number of struct run_s for the task.
- unsigned index

  Index of the actual run.
- unsigned io_time

  IO duration counter.
- enum task_s:: { ... } state

  Tasks state.

4.2.1 Detailed Description

Definition at line 11 of file tasks.h.

4.2.2 Member Enumeration Documentation

4.2.2.1 anonymous enum

Enumerator

ready

blocked

terminated

notready

running

Definition at line 20 of file tasks.h.

```
20 {ready, blocked, terminated, notready, running}
      state;
```

4.2.3 Field Documentation

4.2.3.1 unsigned task_s::duration

The task duration (total)

Definition at line 13 of file tasks.h.

Referenced by ReadTasks().

4.2.3.2 unsigned task_s::index

Index of the actual run.

Definition at line 18 of file tasks.h.

Referenced by ReadTasks(), RunBatch(), RunTimeShare(), and UpdateTasks().

4.2.3.3 unsigned task_s::io_time

IO duration counter.

Definition at line 19 of file tasks.h.

Referenced by ReadTasks().

4.2.3.4 unsigned task_s::nruns

Number of struct run_s for the task.

Definition at line 17 of file tasks.h.

Referenced by DisplayTasks(), and ReadTasks().

4.2.3.5 int task_s::rtime

The task elapsed time (respond time)

Definition at line 15 of file tasks.h.

Referenced by CheckTaskReady(), DisplayStats(), ReadTasks(), RunBatch(), RunTimeShare(), and UpdateTasks().

4.2.3.6 run_t task_s::run

Array of run_s.

Definition at line 16 of file tasks.h.

Referenced by ReadTasks(), RunBatch(), RunTimeShare(), and UpdateTasks().

4.2.3.7 unsigned task_s::start

The task arrival time.

Definition at line 12 of file tasks.h.

Referenced by ReadTasks().

4.2.3.8 enum { ... } task_s::state

Tasks state.

Referenced by BatchResched(), CheckTaskReady(), ReadTasks(), RunBatch(), RunTimeShare(), TimeShareResched(), and UpdateTasks().

4.2.3.9 int task_s::utime

The task user time.

Definition at line 14 of file tasks.h.

Referenced by DisplayStats(), ReadTasks(), RunBatch(), and RunTimeShare().

The documentation for this struct was generated from the following file:

- tasks.h

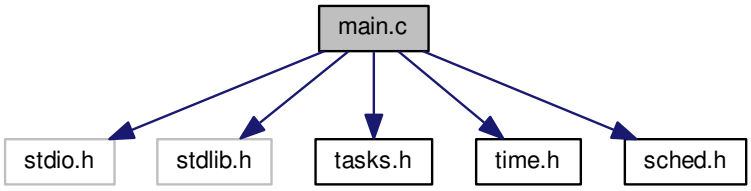
Chapter 5

File Documentation

5.1 main.c File Reference

```
#include <stdio.h>
#include <stdlib.h>
#include "tasks.h"
#include "time.h"
#include "sched.h"
```

Include dependency graph for main.c:



Macros

- #define VERSION "1.1"

Functions

- int main (int argc, char ∗argv[])

Variables

- task_t tasks

  The set of tasks.
- unsigned quantum

  The time slice used for time sharing.

5.1.1 Macro Definition Documentation

5.1.1.1 #define VERSION "1.1"

Definition at line 12 of file main.c.

Referenced by main().

5.1.2 Function Documentation

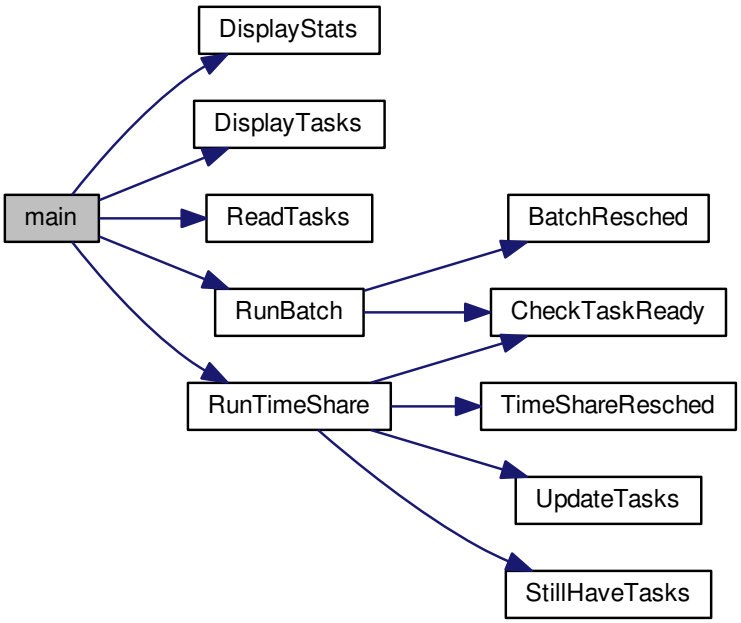
5.1.2.1 int main (int *argc,* char * *argv[]*)

Definition at line 17 of file main.c.

References DisplayStats(), DisplayTasks(), quantum, ReadTasks(), RunBatch(), RunTimeShare(), and VERSION.

```
18 {
19     unsigned n;
20     unsigned runmode;
21     unsigned time;
22     unsigned idle=0;
23 
24     printf("Tasks simulator v%s: ", VERSION);
25 
26     runmode = atoi(argv[1]);
27     switch (runmode)
28     {
29     case 1:
30       printf("BATCH PROCESSING\n\n");
31       n = ReadTasks(argv[2]);               // Read task file
32       DisplayTasks(tasks, n);               // Display available tasks
33       time = RunBatch(tasks, n, &idle);     // Run task in batch processing
34       break;
35     case 2:
36       printf("TIME SHARING\n\n");
37       quantum = atoi(argv[2]);
38       n = ReadTasks(argv[3]);
39       DisplayTasks(tasks, n);
40       time = RunTimeShare(tasks,quantum,n, &idle);          // Run tasks in time
     sharing
41       break;
42     }
43     DisplayStats(tasks, n, time, idle);       // Display tasks statistics after execution
44     return(0);
45 }
```

Here is the call graph for this function:



5.1.3 Variable Documentation

5.1.3.1 unsigned quantum

The time slice used for time sharing.

Definition at line 10 of file time.c.

Referenced by main().

5.1.3.2 task_t tasks

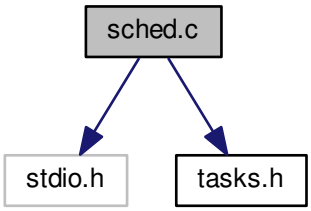
The set of tasks.

Definition at line 16 of file tasks.c.

5.2 sched.c File Reference

```
#include <stdio.h>
#include "tasks.h"
```

Include dependency graph for sched.c:



Functions

- int BatchResched (task_t tasks, int current_task, unsigned n)

  Choose next task in batch processing
  The next task in ready state in chosen.
- unsigned RunBatch (task_t tasks, int n, unsigned *idle)

  Runs the tasks list in batch processing
  Each task is terminated before running the next one. There is no preemption.
  This function is executed at each CPU cycle.
- int TimeShareResched (task_t tasks, int current_task, unsigned n)

  Chose next task to execute (shechdule) in time sharing.
  Uses a simple Round Robin policy without preemption, and without priorities. The function is called at end of quantum or at IO.
- unsigned RunTimeShare (task_t tasks, unsigned quantum, unsigned n, unsigned *idle)

  Runs the tasks in time sharing.
  The function is called a each CPU cycle
  .

5.2.1 Function Documentation

5.2.1.1 int BatchResched (task_t *tasks*, int *current_task*, unsigned *n*)

Choose next task in batch processing

The next task in ready state in chosen.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The current_task index
in	*n*	The number of tasks

Definition at line 18 of file sched.c.

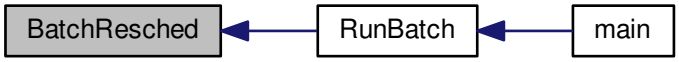
References task_s::state.

Referenced by RunBatch().

```
19 {
20     unsigned next;
21     unsigned count=0;
22
23     for(next=0; next<n; next++)
```

```
24          if (tasks[next].state == ready) // This is the new task
25          {
26              tasks[next].state = running;
27              return(next);
28          } else if (tasks[next].state == notready) count++;
29      if (count != 0) return(-2); // Some task are not yet ready
30          else return(-1); // There is no more task
31  }
```

Here is the caller graph for this function:



5.2.1.2 unsigned RunBatch (task_t *tasks*, int *n*, unsigned * *idle*)

Runs the tasks list in batch processing

Each task is terminated before running the next one. There is no preemption.

This function is executed at each CPU cycle.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The task being executed
in	*idle*	The CPU idle time

Returns

The total running time

Definition at line 43 of file sched.c.

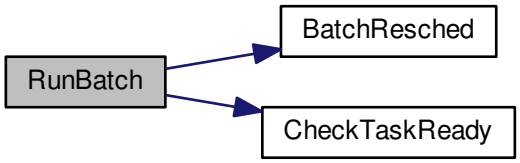
References BatchResched(), CheckTaskReady(), task_s::index, run_s::io, task_s::rtime, run_s::run, task_s::run, task_s::state, and task_s::utime.

Referenced by main().

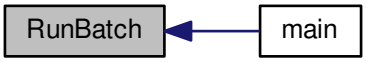
```
44  {
45      unsigned time;
46      unsigned i;
47      unsigned current_task=0;
48      unsigned rdy=0;
49      unsigned index;
50
51      time=0; // starting time;
52      *idle = 0; // cpu idel time
53
54
55      while(1)
56      {
57          rdy = CheckTaskReady(tasks, time, n);
58          if (rdy > 0)
59          {
60              // Run the next task
61              current_task = BatchResched(tasks, current_task, n);
62              index = tasks[current_task].index;
63              while( index < tasks[current_task].nruns)
64              {
65                  switch (tasks[current_task].state)
```

```
66                 {
67                 case running:
68                     printf("%d",current_task);
69                     tasks[current_task].utime++;
70                     tasks[current_task].rtime++;
71                     tasks[current_task].run[index].run--;
72                     // updates every other task in ready state
73                     for(i=0; i<n; i++)
74                         if (i != current_task)
75                             if( tasks[i].state == ready) tasks[i].rtime++;
76                     if (tasks[current_task].run[index].run == 0)
77                     {
78                         if (tasks[current_task].run[index].io != 0)
79                         {
80                             // task start I/O
81                             tasks[current_task].state = blocked;
82                             printf("#");
83                             fflush(stdout);
84                         } else
85                             index++;
86                     }
87                     break;
88                 case blocked:
89                     printf("X");
90                     (*idle)++;
91                     tasks[current_task].rtime++;
92                     // updates every other task in ready state
93                     for (i = 0; i < n; i++)
94                         if (i != current_task)
95                             if (tasks[i].state == ready)
96                                 tasks[i].rtime++;
97                     tasks[current_task].run[index].io--;
98                     if (tasks[current_task].run[index].io == 0)
99                     {
100                         // I/O is finished
101                         tasks[current_task].state = running;
102                         // next run
103                         index++;
104                     }
105                     break;
106                 default: break;
107                 }
108                 time++;
109                 rdy = CheckTaskReady(tasks, time, n);
110             }
111             // Task terminated;
112             tasks[current_task].state = terminated;
113             printf("*/");fflush(stdout);
114         }
115         else
116         {
117             // There is no task ready
118             // Check if there are still task to run...
119             current_task = BatchResched(tasks, current_task, n);
120             if (current_task == -1) return(time);
121             else time++;
122         }
123 
124     }
125     return(0);
126 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



5.2.1.3 unsigned RunTimeShare (task_t *tasks*, unsigned *quantum*, unsigned *n*, unsigned * *idle*)

Runs the tasks in time sharing.

The function is called a each CPU cycle

.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The current running task
in	*quantum*	The time slice
in	*idle*	The CPU idle time

Returns

1 If we need to re-schedule (the task ends quantum, IO, or ended)
0 If the tasks still has the CPU

Definition at line 175 of file sched.c.

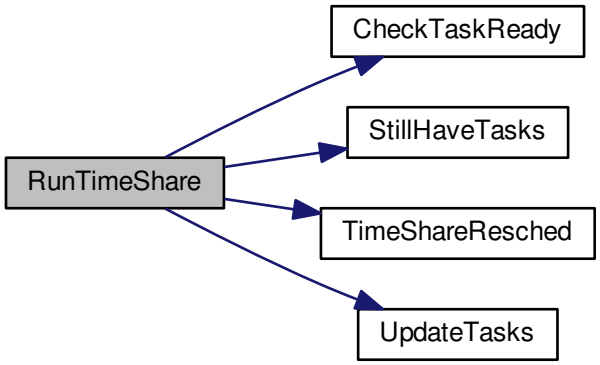
References CheckTaskReady(), task_s::index, task_s::rtime, run_s::run, task_s::run, task_s::state, StillHaveTasks(), TimeShareResched(), UpdateTasks(), and task_s::utime.

Referenced by main().

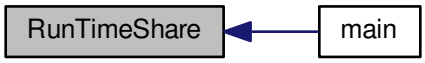
```
176 {
177     unsigned time;
178     unsigned rdy;
179     int current_task=-1;
180     unsigned used_quantum;
181     unsigned index;
182 
183     // Starts time
184     time = 0;
185     *idle = 0;
186 
187     while(1)
188     {
189         rdy = CheckTaskReady(tasks, time, n);
190         if (rdy > 0)
191         {
192             // Run the next task
193             current_task = TimeShareResched(tasks, current_task, n);
194             used_quantum = 0;
195             index = tasks[current_task].index;
196             // Run task for at most one quantum
197             while(used_quantum < quantum)
198             {
199                 printf("%d", current_task);fflush(stdout);
200                 tasks[current_task].utime++;
201                 tasks[current_task].rtime++;
202                 tasks[current_task].run[index].run--;
203                 // updates every other task in ready or blocked states
204                 UpdateTasks(time, n);
205                 // Check if task reached I/O
```

```
206                 if (tasks[current_task].run[index].run == 0)
207                 {
208                     if (tasks[current_task].run[index].io != 0)
209                     {
210                         // task start I/O
211                         printf("#");fflush(stdout);
212                         tasks[current_task].state = blocked;
213                         time++;
214                         break; // exit from while and reschedule tasks (round robin)
215                     }
216                     else
217                     {
218                         // task ends and reschedule
219                         printf("*");fflush(stdout);
220                         tasks[current_task].state = terminated;
221                         time++;
222                         break;
223                     }
224                 }
225                 used_quantum++;
226                 time++;
227                 rdy = CheckTaskReady(tasks, time, n);
228             }
229             // task has used quantum or is in I/O
230             printf("/");fflush(stdout);
231             if (tasks[current_task].state == running) tasks[current_task].
    state = ready;
232         }
233         else
234         {
235             // Wait for tasks to be ready
236             current_task = -1;
237             if (!StillHaveTasks(tasks,n)) return(time);
238             (*idle)++;
239             printf("X"); fflush(stdout);
240             UpdateTasks(time, n);
241             time++;
242             rdy = CheckTaskReady(tasks, time, n);
243         }
244     }
245     return(0);
246 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



5.2.1.4 int TimeShareResched (task_t *tasks*, int *current_task*, unsigned *n*)

Chose next task to execute (shechdule) in time sharing.

Uses a simple Round Robin policy without preemption, and without priorities. The function is called at end of quantum or at IO.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The current task in execution
in	*n*	The number of tasks

Returns

The next task to run
-1 If there is no more tasks
-2 If there are tasks in state notready or blocked and none is available to run

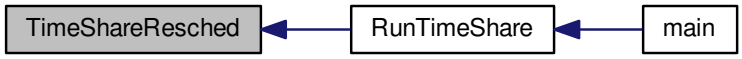
Definition at line 140 of file sched.c.

References task_s::state.

Referenced by RunTimeShare().

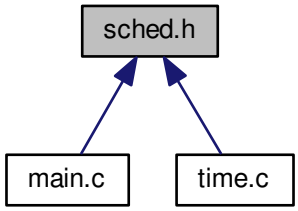
```
141 {
142     unsigned count=0;
143     unsigned wait=0;
144     int index;
145 
146     if (current_task == -2) index = -1;
147         else index = current_task;
148 
149     while( count<n )
150     {
151         index = (index+1)%n;
152         count++;
153         if (tasks[index].state == ready)
154         {
155             tasks[index].state = running;
156             return(index);
157         }
158         if ((tasks[index].state == blocked) || (tasks[index].
      state == notready)) wait++;
159     }
160     if (wait != 0) return(-2);
161         else return(-1);
162 }
```

Here is the caller graph for this function:



5.3 sched.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

- int BatchResched (task_t tasks, int current_task, unsigned n)

  Choose next task in batch processing
  The next task in ready state in chosen.
- unsigned RunBatch (task_t tasks, int n, unsigned *idle)

  Runs the tasks list in batch processing
  Each task is terminated before running the next one. There is no preemption.
  This function is executed at each CPU cycle.
- int TimeShareResched (task_t tasks, int current_task, unsigned n)

  Chose next task to execute (shechdule) in time sharing.
  Uses a simple Round Robin policy without preemption, and without priorities. The function is called at end of quantum or at IO.
- unsigned RunTimeShare (task_t tasks, unsigned quantum, unsigned n, unsigned *idle)

  Runs the tasks in time sharing.
  The function is called a each CPU cycle
  .

5.3.1 Function Documentation

5.3.1.1 int BatchResched (task_t *tasks*, int *current_task*, unsigned *n*)

Choose next task in batch processing

The next task in ready state in chosen.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The current_task index
in	*n*	The number of tasks

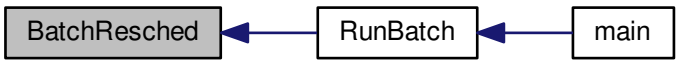
Definition at line 18 of file sched.c.

References task_s::state.

Referenced by RunBatch().

```
19 {
20     unsigned next;
21     unsigned count=0;
22 
23     for(next=0; next<n; next++)
24         if (tasks[next].state == ready) // This is the new task
25         {
26             tasks[next].state = running;
27             return(next);
28         } else if (tasks[next].state == notready) count++;
29     if (count != 0) return(-2); // Some task are not yet ready
30         else return(-1); // There is no more task
31 }
```

Here is the caller graph for this function:



5.3.1.2 unsigned RunBatch (task_t *tasks*, int *n*, unsigned * *idle*)

Runs the tasks list in batch processing

Each task is terminated before running the next one. There is no preemption.

This function is executed at each CPU cycle.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The task being executed
in	*idle*	The CPU idle time

Returns

The total running time

Definition at line 43 of file sched.c.

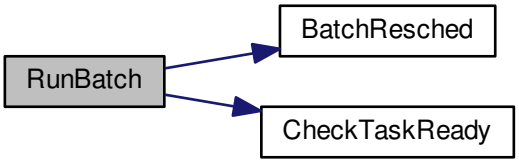
References BatchResched(), CheckTaskReady(), task_s::index, run_s::io, task_s::rtime, run_s::run, task_s::run, task_s::state, and task_s::utime.

Referenced by main().

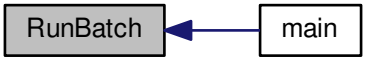
```
44 {
45     unsigned time;
```

```
46      unsigned i;
47      unsigned current_task=0;
48      unsigned rdy=0;
49      unsigned index;
50
51      time=0; // starting time;
52      *idle = 0; // cpu idel time
53
54
55      while(1)
56      {
57          rdy = CheckTaskReady(tasks, time, n);
58          if (rdy > 0)
59          {
60              // Run the next task
61              current_task = BatchResched(tasks, current_task, n);
62              index = tasks[current_task].index;
63              while( index < tasks[current_task].nruns)
64              {
65                  switch (tasks[current_task].state)
66                  {
67                  case running:
68                      printf("%d",current_task);
69                      tasks[current_task].utime++;
70                      tasks[current_task].rtime++;
71                      tasks[current_task].run[index].run--;
72                      // updates every other task in ready state
73                      for(i=0; i<n; i++)
74                          if (i != current_task)
75                              if( tasks[i].state == ready) tasks[i].rtime++;
76                      if (tasks[current_task].run[index].run == 0)
77                      {
78                          if (tasks[current_task].run[index].io != 0)
79                          {
80                              // task start I/O
81                              tasks[current_task].state = blocked;
82                              printf("#");
83                              fflush(stdout);
84                          } else
85                              index++;
86                      }
87                      break;
88                  case blocked:
89                      printf("X");
90                      (*idle)++;
91                      tasks[current_task].rtime++;
92                      // updates every other task in ready state
93                      for (i = 0; i < n; i++)
94                          if (i != current_task)
95                              if (tasks[i].state == ready)
96                                  tasks[i].rtime++;
97                      tasks[current_task].run[index].io--;
98                      if (tasks[current_task].run[index].io == 0)
99                      {
100                          // I/O is finished
101                          tasks[current_task].state = running;
102                          // next run
103                          index++;
104                      }
105                      break;
106                  default: break;
107                  }
108                  time++;
109                  rdy = CheckTaskReady(tasks, time, n);
110              }
111              // Task terminated;
112              tasks[current_task].state = terminated;
113              printf("*/");fflush(stdout);
114          }
115          else
116          {
117              // There is no task ready
118              // Check if there are still task to run...
119              current_task = BatchResched(tasks, current_task, n);
120              if (current_task == -1) return(time);
121              else time++;
122          }
123
124      }
125      return(0);
126  }
```

Here is the call graph for this function:



Here is the caller graph for this function:



5.3.1.3 unsigned RunTimeShare (task_t *tasks*, unsigned *quantum*, unsigned *n*, unsigned * *idle*)

Runs the tasks in time sharing.

The function is called a each CPU cycle

.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The current running task
in	*quantum*	The time slice
in	*idle*	The CPU idle time

Returns

1 If we need to re-schedule (the task ends quantum, IO, or ended)
0 If the tasks still has the CPU

Definition at line 175 of file sched.c.

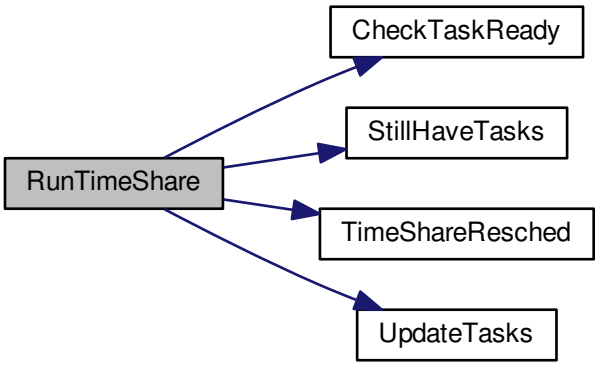
References CheckTaskReady(), task_s::index, task_s::rtime, run_s::run, task_s::run, task_s::state, StillHaveTasks(), TimeShareResched(), UpdateTasks(), and task_s::utime.

Referenced by main().

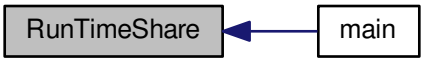
```
176 {
177     unsigned time;
178     unsigned rdy;
179     int current_task=-1;
180     unsigned used_quantum;
181     unsigned index;
```

```
182
183     // Starts time
184     time = 0;
185     *idle = 0;
186
187     while(1)
188     {
189         rdy = CheckTaskReady(tasks, time, n);
190         if (rdy > 0)
191         {
192             // Run the next task
193             current_task = TimeShareResched(tasks, current_task, n);
194             used_quantum = 0;
195             index = tasks[current_task].index;
196             // Run task for at most one quantum
197             while(used_quantum < quantum)
198             {
199                 printf("%d", current_task);fflush(stdout);
200                 tasks[current_task].utime++;
201                 tasks[current_task].rtime++;
202                 tasks[current_task].run[index].run--;
203                 // updates every other task in ready or blocked states
204                 UpdateTasks(time, n);
205                 // Check if task reached I/O
206                 if (tasks[current_task].run[index].run == 0)
207                 {
208                     if (tasks[current_task].run[index].io != 0)
209                     {
210                         // task start I/O
211                         printf("#");fflush(stdout);
212                         tasks[current_task].state = blocked;
213                         time++;
214                         break; // exit from while and reschedule tasks (round robin)
215                     }
216                     else
217                     {
218                         // task ends and reschedule
219                         printf("*");fflush(stdout);
220                         tasks[current_task].state = terminated;
221                         time++;
222                         break;
223                     }
224                 }
225                 used_quantum++;
226                 time++;
227                 rdy = CheckTaskReady(tasks, time, n);
228             }
229             // task has used quantum or is in I/O
230             printf("/");fflush(stdout);
231             if (tasks[current_task].state == running) tasks[current_task].
    state = ready;
232         }
233         else
234         {
235             // Wait for tasks to be ready
236             current_task = -1;
237             if (!StillHaveTasks(tasks,n)) return(time);
238             (*idle)++;
239             printf("X"); fflush(stdout);
240             UpdateTasks(time, n);
241             time++;
242             rdy = CheckTaskReady(tasks, time, n);
243         }
244     }
245     return(0);
246 }
```

Here is the call graph for this function:



Here is the caller graph for this function:



5.3.1.4 int TimeShareResched (task_t *tasks*, int *current_task*, unsigned *n*)

Chose next task to execute (shechdule) in time sharing.

Uses a simple Round Robin policy without preemption, and without priorities. The function is called at end of quantum or at IO.

Parameters

in	*tasks*	The tasks list
in	*current_task*	The current task in execution
in	*n*	The number of tasks

Returns

The next task to run
-1 If there is no more tasks
-2 If there are tasks in state notready or blocked and none is available to run

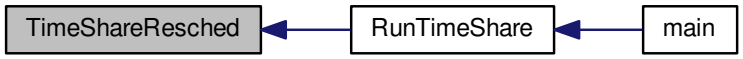
Definition at line 140 of file sched.c.

References task_s::state.

Referenced by RunTimeShare().

```
141 {
142     unsigned count=0;
143     unsigned wait=0;
144     int index;
145 
146     if (current_task == -2) index = -1;
147         else index = current_task;
148 
149     while( count<n )
150     {
151         index = (index+1)%n;
152         count++;
153         if (tasks[index].state == ready)
154         {
155             tasks[index].state = running;
156             return(index);
157         }
158         if ((tasks[index].state == blocked) || (tasks[index].
      state == notready)) wait++;
159     }
160     if (wait != 0) return(-2);
161         else return(-1);
162 }
```

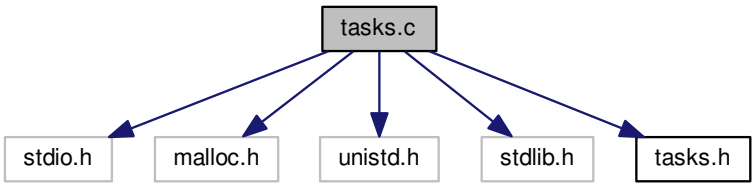
Here is the caller graph for this function:



5.4 tasks.c File Reference

```
#include <stdio.h>
#include <malloc.h>
#include <unistd.h>
#include <stdlib.h>
#include "tasks.h"
```

Include dependency graph for tasks.c:



Macros

- #define TASKS_C

Functions

- unsigned int ReadTasks (char *filename)

  Read tasks from file. File is formated as follow: Number of tasks arrival IO_NUMBER IO_TIME:IO_DURATION ...
- void DisplayTasks (task_t tasks, unsigned n)

  Display the task list as read from file.
- unsigned CheckTaskReady (task_t tasks, unsigned time, unsigned n)

  Checks for new tasks arrival. Tasks have a start time Put all the new tasks in ready state.
- unsigned StillHaveTasks (task_t tasks, unsigned n)
- void UpdateTasks (unsigned time, unsigned n)

  Update task statistics and IOs at each CPU cycle Update the elapsed time of blocked and ready tasks, and simulate IOs duration.

Variables

- unsigned int ntasks =0

  The number of tasks.
- task_t tasks

  The set of tasks.

5.4.1 Macro Definition Documentation

5.4.1.1 #define TASKS_C

Definition at line 3 of file tasks.c.

5.4.2 Function Documentation

5.4.2.1 unsigned CheckTaskReady (task_t *tasks*, unsigned *time*, unsigned *n*)

Checks for new tasks arrival. Tasks have a start time Put all the new tasks in ready state.

Parameters

in	*tasks*	The tasks list
in	*time*	The actual CPU time
in	*n*	The number of tasks

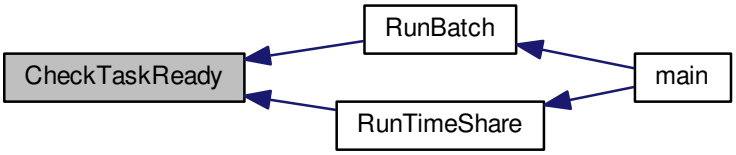
Definition at line 105 of file tasks.c.

References task_s::rtime, and task_s::state.

Referenced by RunBatch(), and RunTimeShare().

```
106 {
107     unsigned i;
108     unsigned int rdy=0;
109 
110     for (i=0; i<n; i++)
111     {
112         if (tasks[i].start == time)
113         {
114             tasks[i].state = ready;
115             tasks[i].rtime = 0;
116         }
117         if ((tasks[i].state == ready) || (tasks[i].state ==
    running)) rdy++;
118     }
119     return(rdy);
120 }
```

Here is the caller graph for this function:



5.4.2.2 void DisplayTasks (task_t *tasks*, unsigned *n*)

Display the task list as read from file.

Parameters

in	*tasks*	The tasks list
in	*n*	The number of tasks

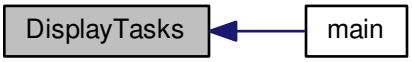
Definition at line 78 of file tasks.c.

References task_s::nruns.

Referenced by main().

```
79 {
80     unsigned int i,j;
81 
82     for(i=0; i<n; i++)
83     {
84         printf("Task %d: start at: %d, duration: %d\n", i, tasks[i].start, tasks[i].
       duration);
85         printf("\tIOs at: ");
86         for(j=0; j<tasks[i].nruns; j++)
87         {
88             if (tasks[i].run[j].io == 0)
89                 printf("%d:END", tasks[i].run[j].run);
90             else
91                 printf("%d:%d  ", tasks[i].run[j].run, tasks[i].run[j].io);
92         }
93         printf("\n");
94     }
95 }
```

Here is the caller graph for this function:



5.4.2.3 unsigned int ReadTasks (char * *filename*)

Read tasks from file. File is formated as follow: Number of tasks arrival IO_NUMBER IO_TIME:IO_DURATION ...

Parameters

in	*filename*	File name

Returns

The number of tasks

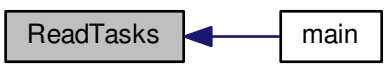
Definition at line 27 of file tasks.c.

References task_s::duration, task_s::index, run_s::io, task_s::io_time, task_s::nruns, task_s::rtime, run_s::run, task_s::run, task_s::start, task_s::state, and task_s::utime.

Referenced by main().

```
28 {
29     FILE *file;
30     unsigned int n;
31     unsigned int i,j;
32     unsigned int start,duration,ios,sum;
33     run_t run;
34     unsigned int iot, iod;
35 
36     file = fopen(filename, "r");
37     if (file == NULL)
38     {
39         perror("Error lectura tareas");
40         exit(1);
41     }
42     fscanf(file, "%d\n",&n);
43     tasks = (task_t)malloc(n*sizeof(struct task_s));
44     for(i=0;i<n;i++)
45     {
46         sum = 0;
47         fscanf(file,"%d %d %d ", &start, &duration, &ios);
48         tasks[i].start = start;
49         tasks[i].duration = duration;
50         tasks[i].utime = 0;
51         tasks[i].rtime = 0;
52         tasks[i].nruns = ios+1;
53         tasks[i].index = 0;
54         tasks[i].io_time = 0;
55         tasks[i].state = notready;
56         run = (run_t)malloc(sizeof(struct run_s)*(ios+1));
57         tasks[i].run = run;
58         for(j=0; j<ios; j++)
59         {
60             fscanf(file, "%d:%d", &iot, &iod);
61             run[j].run = iot;
62             run[j].io = iod;
63             sum += iot;
64         }
65         run[ios].run = duration - sum;
66         run[ios].io = 0;
67         fscanf(file, "\n");
68     }
69     fclose(file);
70     return(n);
71 }
```

Here is the caller graph for this function:



5.4.2.4 unsigned StillHaveTasks (task_t *tasks*, unsigned *n*)

Definition at line 130 of file tasks.c.

Referenced by RunTimeShare().

```
131 {
132     unsigned i;
133     unsigned count=0;
134 
135     for(i=0; i<n; i++)
136         if ((tasks[i].state == notready) || (tasks[i].state ==
      blocked)) count++;
137     return(count);
138 }
```

Here is the caller graph for this function:



5.4.2.5 void UpdateTasks (unsigned *time*, unsigned *n*)

Update task statistics and IOs at each CPU cycle Update the elapsed time of blocked and ready tasks, and simulate IOs duration.

Parameters

in	*time*	The actual CPU time
in	*n*	The number of tasks

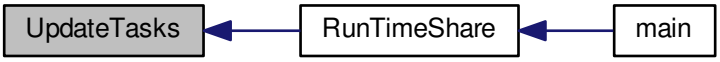
Definition at line 147 of file tasks.c.

References task_s::index, run_s::io, task_s::rtime, task_s::run, and task_s::state.

Referenced by RunTimeShare().

```
148 {
149     unsigned i;
150 
151     for(i=0; i<n; i++)
152     {
153         switch( tasks[i].state)
154         {
155         case ready:
156             tasks[i].rtime++;
157             break;
158         case blocked:
159             tasks[i].rtime++;
160             tasks[i].run[tasks[i].index].io--;
161             if (tasks[i].run[tasks[i].index].io == 0)
162             {
163                 tasks[i].index++;
164                 tasks[i].state = ready;
165             }
166             break;
167         case notready:
168         case running:
169         default: break;
170         }
171     }
172 }
```

Here is the caller graph for this function:



5.4.3 Variable Documentation

5.4.3.1 unsigned int ntasks =0

The number of tasks.

Definition at line 13 of file tasks.c.

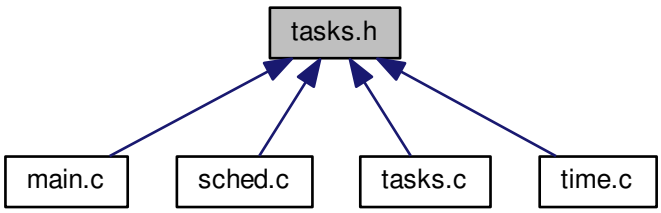
5.4.3.2 task_t tasks

The set of tasks.

Definition at line 16 of file tasks.c.

5.5 tasks.h File Reference

This graph shows which files directly or indirectly include this file:



Data Structures

- struct run_s
- struct task_s

Typedefs

- typedef struct run_s * run_t
- typedef struct task_s * task_t

The task list.

Enumerations

- enum sched_t { batch, timeshare }

Functions

- unsigned int ReadTasks (char *filename)

  Read tasks from file. File is formated as follow: Number of tasks arrival IO_NUMBER IO_TIME:IO_DURATION ...
- void DisplayTasks (task_t tasks, unsigned n)

  Display the task list as read from file.
- unsigned CheckTaskReady (task_t tasks, unsigned time, unsigned n)

  Checks for new tasks arrival. Tasks have a start time Put all the new tasks in ready state.
- void UpdateTasks (unsigned time, unsigned n)

  Update task statistics and IOs at each CPU cycle Update the elapsed time of blocked and ready tasks, and simulate IOs duration.
- unsigned StillHaveTasks (task_t tasks, unsigned n)

5.5.1 Typedef Documentation

5.5.1.1 typedef struct run_s* run_t

Definition at line 9 of file tasks.h.

5.5.1.2 typedef struct task_s* task_t

The task list.

Definition at line 23 of file tasks.h.

5.5.2 Enumeration Type Documentation

5.5.2.1 enum sched_t

Enumerator

batch

timeshare

Definition at line 24 of file tasks.h.

```
24 {batch, timeshare} sched_t;
```

5.5.3 Function Documentation

5.5.3.1 unsigned CheckTaskReady (task_t *tasks*, unsigned *time*, unsigned *n*)

Checks for new tasks arrival. Tasks have a start time Put all the new tasks in ready state.

Parameters

in	*tasks*	The tasks list
in	*time*	The actual CPU time
in	*n*	The number of tasks

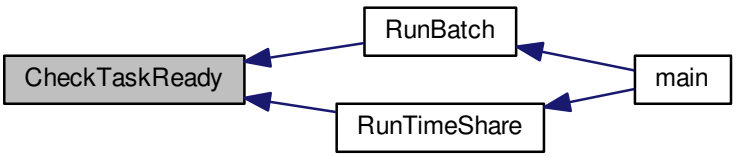
Definition at line 105 of file tasks.c.

References task_s::rtime, and task_s::state.

Referenced by RunBatch(), and RunTimeShare().

```
106 {
107     unsigned i;
108     unsigned int rdy=0;
109 
110     for (i=0; i<n; i++)
111     {
112         if (tasks[i].start == time)
113         {
114             tasks[i].state = ready;
115             tasks[i].rtime = 0;
116         }
117         if ((tasks[i].state == ready) || (tasks[i].state ==
      running)) rdy++;
118     }
119     return(rdy);
120 }
```

Here is the caller graph for this function:



5.5.3.2 void DisplayTasks (task_t *tasks*, unsigned *n*)

Display the task list as read from file.

Parameters

in	*tasks*	The tasks list
in	*n*	The number of tasks

Definition at line 78 of file tasks.c.

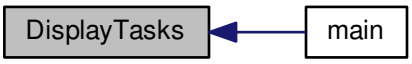
References task_s::nruns.

Referenced by main().

```
79 {
80     unsigned int i,j;
81 
82     for(i=0; i<n; i++)
83     {
84         printf("Task %d: start at: %d, duration: %d\n", i, tasks[i].start, tasks[i].
      duration);
85         printf("\tIOs at: ");
86         for(j=0; j<tasks[i].nruns; j++)
```

```
87          {
88              if (tasks[i].run[j].io == 0)
89                  printf("%d:END", tasks[i].run[j].run);
90              else
91                  printf("%d:%d  ", tasks[i].run[j].run, tasks[i].run[j].io);
92          }
93          printf("\n");
94      }
95  }
```

Here is the caller graph for this function:



5.5.3.3 unsigned int ReadTasks (char * *filename*)

Read tasks from file. File is formated as follow: Number of tasks arrival IO_NUMBER IO_TIME:IO_DURATION ...

Parameters

in	*filename*	File name

Returns

The number of tasks

Definition at line 27 of file tasks.c.

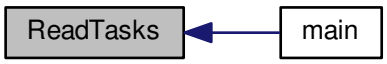
References task_s::duration, task_s::index, run_s::io, task_s::io_time, task_s::nruns, task_s::rtime, run_s::run, task_s::run, task_s::start, task_s::state, and task_s::utime.

Referenced by main().

```
28  {
29      FILE *file;
30      unsigned int n;
31      unsigned int i,j;
32      unsigned int start,duration,ios,sum;
33      run_t run;
34      unsigned int iot, iod;
35
36      file = fopen(filename, "r");
37      if (file == NULL)
38      {
39          perror("Error lectura tareas");
40          exit(1);
41      }
42      fscanf(file, "%d\n",&n);
43      tasks = (task_t)malloc(n*sizeof(struct task_s));
44      for(i=0;i<n;i++)
45      {
46          sum = 0;
47          fscanf(file,"%d %d %d ", &start, &duration, &ios);
48          tasks[i].start = start;
49          tasks[i].duration = duration;
50          tasks[i].utime = 0;
51          tasks[i].rtime = 0;
52          tasks[i].nruns = ios+1;
53          tasks[i].index = 0;
54          tasks[i].io_time = 0;
55          tasks[i].state = notready;
56          run = (run_t)malloc(sizeof(struct run_s)*(ios+1));
```

```
57          tasks[i].run = run;
58          for(j=0; j<ios; j++)
59          {
60              fscanf(file, "%d:%d", &iot, &iod);
61              run[j].run = iot;
62              run[j].io = iod;
63              sum += iot;
64          }
65          run[ios].run = duration - sum;
66          run[ios].io = 0;
67          fscanf(file, "\n");
68      }
69      fclose(file);
70      return(n);
71  }
```

Here is the caller graph for this function:



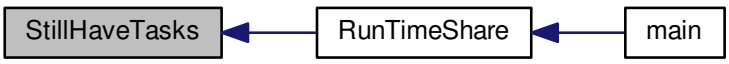
5.5.3.4 unsigned StillHaveTasks (task_t *tasks*, unsigned *n*)

Definition at line 130 of file tasks.c.

Referenced by RunTimeShare().

```
131 {
132     unsigned i;
133     unsigned count=0;
134 
135     for(i=0; i<n; i++)
136         if ((tasks[i].state == notready) || (tasks[i].state ==
      blocked)) count++;
137     return(count);
138 }
```

Here is the caller graph for this function:



5.5.3.5 void UpdateTasks (unsigned *time*, unsigned *n*)

Update task statistics and IOs at each CPU cycle Update the elapsed time of blocked and ready tasks, and simulate IOs duration.

Parameters

in	*time*	The actual CPU time
in	*n*	The number of tasks

Definition at line 147 of file tasks.c.

References task_s::index, run_s::io, task_s::rtime, task_s::run, and task_s::state.

Referenced by RunTimeShare().

```
148 {
149     unsigned i;
150 
151     for(i=0; i<n; i++)
152     {
153         switch( tasks[i].state)
154         {
155         case ready:
156             tasks[i].rtime++;
157             break;
158         case blocked:
159             tasks[i].rtime++;
160             tasks[i].run[tasks[i].index].io--;
161             if (tasks[i].run[tasks[i].index].io == 0)
162             {
163                 tasks[i].index++;
164                 tasks[i].state = ready;
165             }
166             break;
167         case notready:
168         case running:
169         default: break;
170         }
171     }
172 }
```

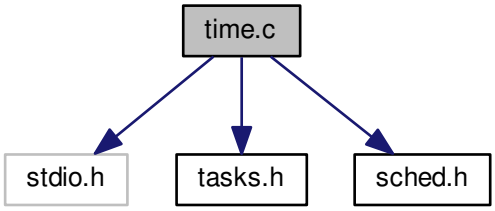
Here is the caller graph for this function:



5.6 time.c File Reference

```
#include <stdio.h>
#include "tasks.h"
#include "sched.h"
```

Include dependency graph for time.c:



Macros

- #define TIME_C

Enumerations

- enum { true, false }

Functions

- void DisplayStats (task_t tasks, unsigned n, unsigned total, unsigned idle)

  Displays the tasks statistics when simulation ends:

  .

Variables

- unsigned cpu_time =0

  The CPU virtual time.
- unsigned quantum

  The time slice used for time sharing.
- enum { ... } cpu_end =false

  Terminates simulation?

5.6.1 Macro Definition Documentation

5.6.1.1 #define TIME_C

Definition at line 3 of file time.c.

5.6.2 Enumeration Type Documentation

5.6.2.1 anonymous enum

Enumerator

true

false

Definition at line 11 of file time.c.

```
11 {true, false} cpu_end=false;
```

5.6.3 Function Documentation

5.6.3.1 void DisplayStats (task_t *tasks*, unsigned *n*, unsigned *total*, unsigned *idle*)

Displays the tasks statistics when simulation ends:

.

- Total execution time
- CPU time
- Elapsed time (respond time)

Parameters

in	*tasks*	The tasks list
in	*n*	The number of tasks
in	*total*	The global execution time
in	*idle*	The CPU idle time

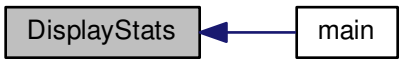
Definition at line 23 of file time.c.

References task_s::rtime, and task_s::utime.

Referenced by main().

```
24 {
25     unsigned i;
26 
27     printf("\n\nTasks: %d, Total execution time: %d\n", n, total);
28     printf("Total CPU idle time: %d (%2.2f%%)\n", idle, ((float)idle/(float)total)*100);
29     for(i=0; i<n; i++)
30     {
31         printf("Task %d, CPU time: %d, Elapsed: %d\n", i, tasks[i].utime, tasks[i].
      rtime);
32     }
33 }
```

Here is the caller graph for this function:



5.6.4 Variable Documentation

5.6.4.1 enum { ... } cpu_end

Terminates simulation?

5.6.4.2 unsigned cpu_time =0

The CPU virtual time.

Definition at line 9 of file time.c.

5.6.4.3 unsigned quantum

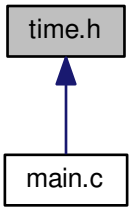
The time slice used for time sharing.

Definition at line 10 of file time.c.

Referenced by main().

5.7 time.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

- void DisplayStats (task_t tasks, unsigned n, unsigned total, unsigned idle)

  Displays the tasks statistics when simulation ends:

  .

5.7.1 Function Documentation

5.7.1.1 void DisplayStats (task_t *tasks,* unsigned *n,* unsigned *total,* unsigned *idle*)

Displays the tasks statistics when simulation ends:

.

- Total execution time
- CPU time
- Elapsed time (respond time)

Parameters

in	*tasks*	The tasks list
in	*n*	The number of tasks
in	*total*	The global execution time
in	*idle*	The CPU idle time

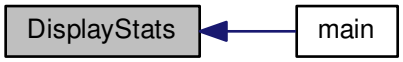
Definition at line 23 of file time.c.

References task_s::rtime, and task_s::utime.

Referenced by main().

```
24 {
25     unsigned i;
26 
27     printf("\n\nTasks: %d, Total execution time: %d\n", n, total);
28     printf("Total CPU idle time: %d (%2.2f%%)\n", idle, ((float)idle/(float)total)*100);
29     for(i=0; i<n; i++)
30     {
31         printf("Task %d, CPU time: %d, Elapsed: %d\n", i, tasks[i].utime, tasks[i].
      rtime);
32     }
33 }
```

Here is the caller graph for this function:



Index

batch
    tasks.h, 33
BatchResched
    sched.c, 14
    sched.h, 20
blocked
    task_s, 9

CheckTaskReady
    tasks.c, 27
    tasks.h, 33
cpu_end
    time.c, 39
cpu_time
    time.c, 39

DisplayStats
    time.c, 39
    time.h, 40
DisplayTasks
    tasks.c, 28
    tasks.h, 34
duration
    task_s, 9

false
    time.c, 38

index
    task_s, 9
io
    run_s, 7
io_time
    task_s, 9

main
    main.c, 12
main.c, 11
    main, 12
    quantum, 13
    tasks, 13
    VERSION, 12

notready
    task_s, 9
nruns
    task_s, 9
ntasks
    tasks.c, 32

quantum
    main.c, 13
    time.c, 40

ReadTasks
    tasks.c, 28
    tasks.h, 35
ready
    task_s, 9
rtime
    task_s, 9
run
    run_s, 7
    task_s, 9
run_s, 7
    io, 7
    run, 7
run_t
    tasks.h, 33
RunBatch
    sched.c, 15
    sched.h, 21
RunTimeShare
    sched.c, 17
    sched.h, 23
running
    task_s, 9

sched.c, 13
    BatchResched, 14
    RunBatch, 15
    RunTimeShare, 17
    TimeShareResched, 19
sched.h, 20
    BatchResched, 20
    RunBatch, 21
    RunTimeShare, 23
    TimeShareResched, 25
sched_t
    tasks.h, 33
start
    task_s, 10
state
    task_s, 10
StillHaveTasks
    tasks.c, 30
    tasks.h, 36

TASKS_C
    tasks.c, 27
TIME_C

time.c, 38
task_s
blocked, 9
notready, 9
ready, 9
running, 9
terminated, 9
task_s, 8
duration, 9
index, 9
io_time, 9
nruns, 9
rtime, 9
run, 9
start, 10
state, 10
utime, 10
task_t
tasks.h, 33
tasks
main.c, 13
tasks.c, 32
tasks.h
batch, 33
timeshare, 33
tasks.c, 26
CheckTaskReady, 27
DisplayTasks, 28
ntasks, 32
ReadTasks, 28
StillHaveTasks, 30
TASKS_C, 27
tasks, 32
UpdateTasks, 31
tasks.h, 32
CheckTaskReady, 33
DisplayTasks, 34
ReadTasks, 35
run_t, 33
sched_t, 33
StillHaveTasks, 36
task_t, 33
UpdateTasks, 36
terminated
task_s, 9
time.c
false, 38
true, 38
time.c, 37
cpu_end, 39
cpu_time, 39
DisplayStats, 39
quantum, 40
TIME_C, 38
time.h, 40
DisplayStats, 40
TimeShareResched
sched.c, 19
sched.h, 25
timeshare
tasks.h, 33
true
time.c, 38

UpdateTasks
tasks.c, 31
tasks.h, 36
utime
task_s, 10

VERSION
main.c, 12